

Build & Run Your MCP Server

From localhost to deploy — steps, tips, and the data layer each use case needs

Client: Participant Hands-On Guide

Trainer: Advanced Agentic AI Bootcamp

Date: Case Studies 1–3



PART 1

What You're Building

An MCP server exposes your tools to any agent over a standard protocol. You'll run it on localhost, connect a client, then bundle and deploy.



Server + Client on Localhost

The MCP server

- A FastAPI app that exposes tools to agents
- Each tool = a typed endpoint (auth + validate + audit)
- Runs as a uvicorn process on a port (HTTP transport)
- Auto-docs at /docs — your fastest sanity check
- Build HTTP transport from day one so deploy is seamless

The client

- Your agent code (CrewAI / LangGraph / Agents SDK)
- Calls the MODEL via the LiteLLM gateway
- Calls TOOLS via your MCP server URL
- Both targets are just URLs in your .env
- Two processes on localhost: server + client



PART 2

Run It on Localhost

The inner dev loop: start the server, point the client at it, smoke-test, iterate.



Start the MCP Server

From your project folder:

WSL / terminal

```
cd ~/projects/philips-agent-lab

# dev mode: auto-reload on file changes
uv run uvicorn server:app \
    --host 127.0.0.1 --port 8000 --reload

# server now live at http://127.0.0.1:8000
# open the auto-generated tool docs:
#   http://127.0.0.1:8000/docs
```

Tips

- --reload restarts on save — dev only, never in production
- 127.0.0.1 = local-only; you'll switch to 0.0.0.0 to deploy
- /docs lists every tool — confirm your wiring here first
- Keep this terminal running; open a second one for the client



Prove a Tool Works Before Wiring the Agent

Hit the tool directly with curl — fastest way to catch auth or schema bugs:

second terminal

```
curl http://127.0.0.1:8000/tools/lookup \  
  -H "Authorization: Bearer <dev-token>" \  
  -H "Content-Type: application/json" \  
  -d '{"query":"test","scope":"public"}'
```

```
# clean JSON back → server + auth OK  
# 401 → token/auth wrong  
# 422 → request schema mismatch
```

Tips

- If curl works, the agent will too — isolate server bugs from agent bugs
- 401 → check your auth layer (Guardrail L2)
- 422 → your Pydantic model rejected the input (L1 working!)
- Test one tool at a time when debugging



Point the Client at Server + Gateway

Both targets live in .env — the client reads them at startup:

```
~/projects/philips-agent-lab/.env  
  
# MODEL — via the LiteLLM gateway  
OPENAI_BASE_URL=http://<gateway>:4000/v1  
OPENAI_API_KEY=<your seat key>  
  
# TOOLS — your own MCP server, local  
MCP_SERVER_URL=http://127.0.0.1:8000  
  
# then, in the second terminal:  
uv run python run_client.py
```

Tips

- Server in terminal 1, client in terminal 2 — both on localhost
- The client never talks to the model directly — always via the gateway
- Edit server → it auto-reloads → re-run client. That's the loop.
- One .env per project keeps context clean



PART 3

Does Your Use Case Need a Data Layer?

Some agents just transform their input. Others must retrieve from a corpus — that needs an embedding model + a store. Decide per use case.



When You Need Embeddings / a Store

You DON'T need one when...

- The agent is handed its input directly
- It structures, drafts, styles, or validates that input
- No search over a larger body of knowledge
- → Plain MCP tools are enough

You DO need one when...

- The agent must **FIND** relevant info from many docs
- Semantic search / RAG over a corpus (embeddings + vector DB)
- Reasoning over connected entities / relationships (graph DB)
- → Expose retrieval as an MCP tool all agents can call



Per Use Case — Quick Guide

CS1 · Documentation Automation

Likely YES

Ingest often searches an existing doc corpus → embeddings + vector DB

CS2 · Content Creation

OPTIONAL

Fact-Check / brand-voice may retrieve reference material → embeddings if so

CS3 · Self-Healing System

LIKELY (mature)

Diagnose retrieves similar past incidents / runbooks → vector or graph DB

Confirm against the final Philips use-case inputs — if the data is 'process this one document', embeddings are optional; if it's 'work across our corpus', they're essential.



Embedding Models (Ollama, local CPU)

Run on your 4 vCPU / 16 GB VM

- **nomic-embed-text** — default: ~270MB, 768-dim, fast, strong
- **mxbai-embed-large** — higher quality, 1024-dim, a bit slower
- **bge-m3** — multilingual / hybrid, ~1.2GB
- **all-minilm** — featherweight, max speed, basic quality
- Pull with: `ollama pull nomic-embed-text`

Route it through the gateway

- Ollama exposes `/v1/embeddings` (OpenAI-compatible)
- Add one model entry in LiteLLM — same key, same base URL
- No external API — consistent with the MOM
- Vector dimension must match the model (768 / 1024)
- Decide the model before you populate the store



Vector & Graph Stores

Vector DB — pick one

- **Local, zero-infra:**
- FAISS — in-process, fast, file-backed
- DuckDB (+ vss) — SQL + vectors in one local file
- Chroma — simple embedded vector store
- pgvector — reuse the gateway's existing Postgres
- Cloud free tiers: Qdrant Cloud, Pinecone starter

Graph DB — if you need relationships

- **When entities & connections matter (CS3 incidents)**
- Neo4j Community — install locally, free
- Neo4j AuraDB — free cloud tier
- Expose graph queries as an MCP tool, same pattern
- Only add this if the use case truly needs a graph



Expose Retrieval as an MCP Tool

Don't bake retrieval into each agent — add it once as a tool every agent can call:

server.py — a retrieval tool

```
@app.post('/tools/search')
async def search(req: SearchRequest,      # L1
                 who = Depends(auth)):    # L2
    rate_limit(who)
    vec = embed(req.query)                 # Ollama /v1/embeddings
    hits = vector_db.query(vec, k=5)       # FAISS/pgvector
    audit(who, 'search', req, hits)        # L4
    return SearchResponse(results=hits)    # L3
```

Why this pattern

- One shared capability, not N copies across agents
- Retrieval inherits the same 4 guardrail layers
- Swap FAISS ↔ pgvector ↔ Neo4j behind one interface
- Clean, testable, and teachable



PART 4

Bundle & Deploy

One codebase, three environments.

Containerise, run locally, then ship to a server

— only config and guardrails change.



Bundle with a Dockerfile

Because you used uv + a lockfile, the build is reproducible:

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev
COPY . .
EXPOSE 8000
CMD ["uv", "run", "uvicorn", "server:app", \
    "--host", "0.0.0.0", "--port", "8000"]
```

Two deploy changes

- Bind 0.0.0.0 (not 127.0.0.1) or nothing outside reaches it
- Drop --reload — dev-only
- uv sync --frozen = the exact hash-verified tree you tested
- Build & test: docker build -t philips-mcp:v1 . then docker run -p 8000:8000



Run It on a Server + Harden

Where to run it

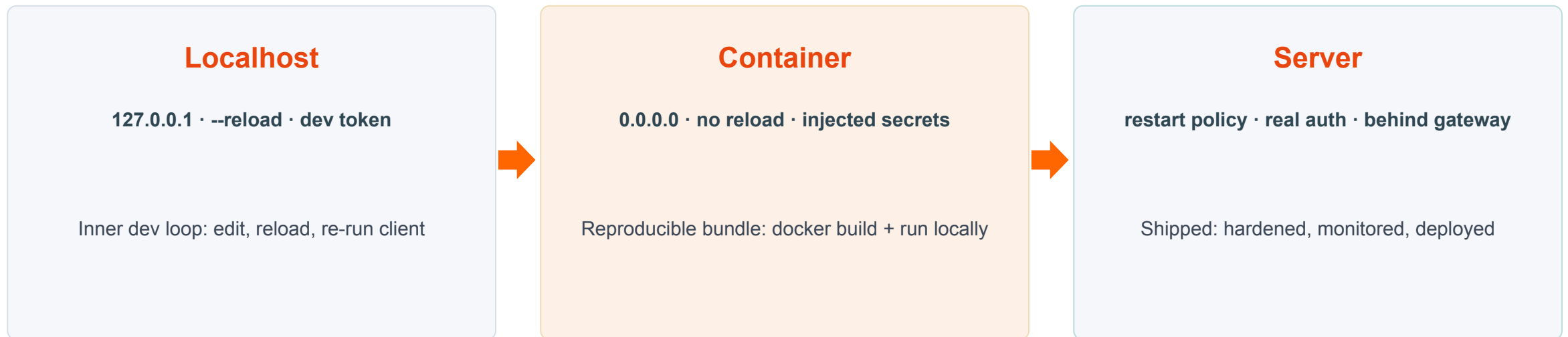
- **Single VM (simplest):**
- `docker run -d --restart=always \`
- `-p 8000:8000 --env-file .env philips-mcp:v1`
- **Managed containers:**
- Azure Container Apps / AWS App Runner (autoscale)
- **docker compose**
- server + deps in one command

Harden before you ship (S7)

- **Secrets injected at runtime — never bake .env into the image or commit keys**
- Auth on every tool endpoint — a server port is the internet, not just you
- `uv sync --frozen` — deploy the tree you tested (supply chain)
- Put it behind the gateway pattern — TLS, rate limits, audit log



One Codebase, Three Environments



Your code doesn't change between environments — only configuration and which guardrails you switch on. The server you smoke-tested on :8000 is what ships.

Build. Run. Ship.

Server on localhost → client connected → retrieval tool if you need it → containerised and deployed. Same code, hardened for the world.
